

ИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №4
по дисциплине «Построение и анализ алгоритмов»
Тема: Поиск подстроки в строке.

Студент гр. 4343

Чулков В.М.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Задание 1

Реализуйте алгоритм КМП и с его помощью для заданных шаблона P ($|P| \leq 25000$) и текста T ($|T| \leq 5000000$) найдите все вхождения P в T .

Вход:

- Первая строка — P
- Вторая строка — T

Выход: индексы начал вхождений P в T , разделённые запятой; если P не входит в T , то вывести -1.

Задание 2

Заданы две строки A ($|A| \leq 5000000$) и B ($|B| \leq 5000000$).

Определить, является ли A циклическим сдвигом B (это значит, что A и B имеют одинаковую длину и A состоит из суффикса B , склеенного с префиксом B). Например, defabc является циклическим сдвигом abcdef.

Вход:

- Первая строка — A
- Вторая строка — B

Выход: Если A является циклическим сдвигом B , то индекс начала строки B в A ; иначе вывести -1. Если возможно несколько сдвигов, вывести первый индекс.

Описание работы алгоритма

1. Поиск всех вхождений шаблона в тексте

Для решения задачи используется алгоритм Кнута — Морриса — Пратта. Он позволяет найти все вхождения шаблона в тексте за линейное время, не возвращаясь назад по тексту при несовпадении символов.

Сначала для шаблона строится префиксный массив. Каждый его элемент показывает длину наибольшего префикса текущей части шаблона, который одновременно является её суффиксом. Этот массив нужен для того, чтобы при несовпадении быстро определить, с какого места шаблона продолжать сравнение.

Во время поиска алгоритм последовательно проходит по тексту и сравнивает его символы с символами шаблона. Переменная текущего состояния хранит количество уже совпавших символов шаблона. Если символы совпадают, это значение увеличивается. Если возникает несовпадение, алгоритм откатывается по префиксному массиву, не начиная поиск заново.

Когда количество совпавших символов становится равно длине шаблона, найдено полное вхождение. Индекс его начала добавляется в список результатов, после чего поиск продолжается дальше, в том числе для возможных пересекающихся вхождений.

Если список найденных позиций не пуст, они выводятся через запятую. Если вхождений нет, выводится -1.

2. Проверка циклического сдвига строк

Для определения, является ли одна строка циклическим сдвигом другой, также используется алгоритм Кнута — Морриса — Пратта.

Сначала проверяется равенство длин строк. Если длины различаются, циклический сдвиг невозможен, и сразу выводится -1.

Основная идея заключается в том, что если строка является циклическим сдвигом другой, то она должна встречаться внутри удвоенного

представления исходной строки. Однако для экономии памяти такая удвоенная строка полностью не создаётся. Вместо этого алгоритм последовательно просматривает исходную строку, а затем её начало, имитируя поиск в циклическом продолжении.

Перед поиском для искомой строки строится префиксный массив. Далее алгоритм КМП сравнивает символы виртуальной строки с символами шаблона. При совпадении увеличивается количество найденных подряд символов, при несовпадении выполняется откат по префиксному массиву.

Если найдено полное совпадение, возвращается индекс начала искомой строки в циклическом представлении. Если после полного просмотра совпадение не найдено, выводится -1.

Описание функций:

Функции задания номер 1:

Сигнатура: `prefix_function(p)`

Назначение:

Строит префиксный массив для строки `p`, который используется в алгоритме Кнута - Морриса - Пратта для быстрого отката при несовпадении символов.

Аргументы:

`p` - строка-шаблон, для которой вычисляется префиксная функция.

Возвращаемое значение:

Список `pi`, где `pi[i]` — длина наибольшего собственного префикса подстроки `p[0:i+1]`, совпадающего с её суффиксом.

Принцип работы:

Функция проходит по строке `p` слева направо и для каждой позиции вычисляет длину максимального совпадающего префикса и суффикса. Переменная `j` хранит длину текущего совпадения. Если текущий символ продолжает совпадение, `j` увеличивается. Если возникает несовпадение, `j` откатывается назад по уже вычисленным значениям массива `pi`. После обработки каждой позиции найденное значение записывается в `pi[i]`.

Сигнатура: `kmp_search(pattern, text)`

Назначение:

Ищет все вхождения строки `pattern` в строке `text` с помощью алгоритма Кнута - Морриса - Пратта.

Аргументы:

`pattern` - искомый шаблон.

`text` - строка, в которой выполняется поиск.

Возвращаемое значение:

Список индексов начала всех вхождений `pattern` в `text`. Если вхождений нет, возвращается пустой список.

Принцип работы:

Сначала для шаблона `pattern` строится префиксный массив с помощью функции `prefix_function`. Затем алгоритм последовательно проходит по символам строки `text`. Переменная `j` хранит количество уже совпавших символов шаблона. При совпадении очередного символа текста с символом шаблона значение `j` увеличивается. При несовпадении `j` откатывается по префиксному массиву, что позволяет не начинать сравнение заново. Когда `j` становится равным длине шаблона, найденное вхождение добавляется в список результатов, после чего выполняется откат для поиска следующих, в том числе пересекающихся, вхождений.

Функции задания номер 2:

Сигнатура: `prefix_function(p)`

Назначение:

Строит префиксный массив для строки `p`, который далее используется при поиске строки в циклическом представлении другой строки.

Аргументы:

`p` - строка-шаблон, для которой необходимо построить префиксную функцию.

Возвращаемое значение:

Список `pi`, где каждый элемент хранит длину максимального собственного префикса, совпадающего с суффиксом соответствующей подстроки.

Принцип работы:

Функция последовательно обрабатывает символы строки `p`. Для каждой позиции она пытается продолжить уже найденное совпадение префикса и суффикса. Если текущий символ совпадает с ожидаемым, длина совпадения

увеличивается. Если символы различаются, происходит откат по массиву pi к более короткому возможному совпадению. Полученное значение записывается в массив pi .

Сигнатура: `kmp_search(pattern, text)`

Назначение:

Проверяет, встречается ли строка `pattern` в циклическом продолжении строки `text`, и возвращает индекс начала найденного сдвига.

Аргументы:

`pattern` - строка, которую нужно найти.

`text` - строка, в циклическом представлении которой выполняется поиск.

Возвращаемое значение:

Целое число - индекс начала строки `pattern` в циклическом представлении `text`. Если совпадение не найдено, возвращается -1.

Принцип работы:

Сначала для строки `pattern` строится префиксный массив. Затем алгоритм выполняет поиск КМП не по реально склеенной строке, а по виртуальной строке `text + text[:-1]`. Первый цикл проходит по исходной строке `text`, второй - по её первым $n - 1$ символам, имитируя циклическое продолжение. Переменная j хранит количество совпавших символов шаблона. При несовпадении выполняется откат по префиксному массиву. Если совпало n символов, функция возвращает индекс начала найденного циклического сдвига. В первом проходе индекс вычисляется стандартно, во втором - с учётом положения во второй части виртуальной строки.

Оценка сложности алгоритма

1. Алгоритм Кнута - Морриса - Пратта. Поиск всех вхождений шаблона в тексте

Временная сложность:

Алгоритм состоит из двух основных этапов. Сначала для шаблона длины m строится префиксный массив. Этот этап выполняется за линейное время, так как каждый символ шаблона обрабатывается один раз, а все откаты по префиксному массиву суммарно также не превышают длину шаблона.

После этого выполняется поиск шаблона в тексте длины n . Алгоритм последовательно проходит по символам текста, не возвращаясь назад. При несовпадении символов происходит откат только по шаблону с использованием заранее построенного префиксного массива. Поэтому каждый символ текста обрабатывается за амортизированное константное время.

Итог: временная сложность - $O(n + m)$, где n - длина текста, m - длина шаблона.

Пространственная сложность:

Основная дополнительная память расходуется на хранение префиксного массива pi длины m . Также используется список `result`, в который записываются индексы всех найденных вхождений. В худшем случае количество вхождений может быть пропорционально длине текста.

Итог: пространственная сложность - $O(m + k)$, где k — количество найденных вхождений. Если не учитывать память под вывод результата, дополнительная память составляет $O(m)$.

2. Алгоритм Кнута - Морриса - Пратта. Проверка циклического сдвига строк

Временная сложность:

Алгоритм сначала проверяет равенство длин строк. Эта операция выполняется за константное время, так как длины строк уже хранятся в памяти.

Затем для строки-шаблона длины n строится префиксный массив. Построение занимает $O(n)$ времени.

После этого выполняется поиск шаблона в циклическом представлении другой строки. При этом строка $\text{text} + \text{text}$ полностью не создаётся. Алгоритм проходит сначала по исходной строке длины n , а затем по первым $n - 1$ символам этой же строки. То есть суммарно просматривается не более $2n - 1$ символов, что асимптотически даёт линейное время.

На каждом шаге при несовпадении используется откат по префиксному массиву, поэтому повторного перебора символов не происходит.

Итог: временная сложность — $O(n)$, где n - длина каждой из строк.

Пространственная сложность:

Дополнительная память расходуется на префиксный массив pi длины n . Важным преимуществом реализации является то, что строка $\text{text} + \text{text}$ не создаётся в памяти полностью, а просматривается виртуально двумя проходами.

Итог: пространственная сложность - $O(n)$.

Вывод

В ходе работы были реализованы два алгоритма на основе метода Кнута - Морриса - Пратта.

Первый алгоритм решает задачу поиска всех вхождений шаблона в тексте. За счёт предварительного построения префиксного массива он выполняет поиск за линейное время $O(n + m)$ и не возвращается назад по тексту при несовпадении символов. Это делает его эффективным для обработки больших строк, где обычный посимвольный поиск мог бы работать значительно медленнее.

Второй алгоритм применяет тот же принцип КМП для проверки циклического сдвига строк. Задача сводится к поиску одной строки в циклическом продолжении другой. При этом реализация не создаёт строку `text + text` целиком, а имитирует её просмотр двумя проходами. Это позволяет сохранить линейную временную сложность $O(n)$ и избежать лишних затрат памяти.

Обе реализации используют префиксный массив для быстрого отката при несовпадении символов. Благодаря этому алгоритмы работают предсказуемо эффективно даже на строках большой длины. Первый алгоритм находит все позиции вхождений шаблона, а второй определяет наличие циклического сдвига и возвращает индекс его начала. Поставленные задачи выполнены полностью.

Приложение

Код программы main1.py:

```
def prefix_function(p):
    pi = [0] * len(p)

    for i in range(1, len(p)):
        j = pi[i - 1]

        print("i =", i, "c =", p[i], "j =", j)

        while j > 0 and p[i] != p[j]:
            print("Откат j:", j, "->", pi[j - 1])
            j = pi[j - 1]

        if p[i] == p[j]:
            j += 1

        pi[i] = j

        print("pi =", pi)
        print()

    print("Префиксный массив =", pi)
    print()

    return pi

def kmp_search(pattern, text):
    pi = prefix_function(pattern)
    result = []

    j = 0

    for i in range(len(text)):
        print("i =", i, "c =", text[i], "j =", j)

        while j > 0 and text[i] != pattern[j]:
            print("Откат j:", j, "->", pi[j - 1])
            j = pi[j - 1]

        if text[i] == pattern[j]:
            j += 1

        print("Новый j =", j)
        print()

        if j == len(pattern):
            found_index = i - len(pattern) + 1
            result.append(found_index)

            print("Найдено вхождение с индекса =", found_index)
            print("result =", result)
```

```

        print("Откат j после найденного вхождения:", j, "->", pi[j
- 1])
        j = pi[j - 1]

        print("Новый j =", j)
        print()

    if not result:
        print("Не найдено")

    return result

if __name__ == "__main__":
    pattern = input()
    text = input()

    result = kmp_search(pattern, text)

    if result:
        print(",".join(map(str, result)))
    else:
        print(-1)

```

Код программы main2.py:

```

def prefix_function(p):
    n = len(p)
    pi = [0] * n
    j = 0

    for i in range(1, n):
        c = p[i]

        print("i =", i, "c =", c, "j =", j)

        while j > 0 and c != p[j]:
            print("Откат j:", j, "->", pi[j - 1])
            j = pi[j - 1]

        if c == p[j]:
            j += 1

        pi[i] = j

        print("pi =", pi)
        print()

    print("Префиксный массив =", pi)
    print()

    return pi

def kmp_search(pattern, text):
    n = len(text)

```

```

if n == 0:
    return 0

pi = prefix_function(pattern)
j = 0

for i in range(n):
    c = text[i]

    print("i =", i, "c =", c, "j =", j)

    while j > 0 and c != pattern[j]:
        print("Откат j:", j, "->", pi[j - 1])
        j = pi[j - 1]

    if c == pattern[j]:
        j += 1

    print("Новый j =", j)
    print()

    if j == n:
        result = i - n + 1
        return result

for i in range(n - 1):
    c = text[i]

    print("i =", i, "c =", c, "j =", j)

    while j > 0 and c != pattern[j]:
        print("Откат j:", j, "->", pi[j - 1])
        j = pi[j - 1]

    if c == pattern[j]:
        j += 1

    print("Новый j =", j)
    print()

    if j == n:
        result = i + 1
        return result

print("Не найдено")
return -1

if __name__ == "__main__":
    text = input()
    pattern = input()

    if len(text) != len(pattern):
        print(-1)
    else:

```

```
result = kmp_search(pattern, text)
print("Результат =", result)
```